Monarchs assignment

Clarissa

06/08/2026

The task here is to load your monarchs csv into R using the `tidyverse` toolkit, calculate and explore the kings' duration of reign with pipes `%>%` in `dplyr` and plot it over time.

Load the monarchs

Make sure to first create an `.Rproj` workspace with a `data/` folder where you place either your own dataset or the provided `monarchs.csv` dataset.

1. Look at the dataset that are you loading and check what its columns are separated by? (hint: open it in plain text editor to see)

List what is the

separator: commas

2. Create a `kings` object in R with the different functions below and inspect the different outputs.

- `read.csv()`
- `read_csv()`
- `read.csv2()`
- `read_csv2()`

```
# FILL IN THE CODE BELOW and review the outputs
monarchs1 <- read.csv("data/monarchs_clean.csv")

monarchs2 <- read_csv("data/monarchs_clean.csv")

monarchs3 <- read.csv2("data/monarchs_clean.csv")

monarchs4 <- read_csv2( "data/monarchs_clean.csv")
```

Answer: 1. Which of these functions is a `tidyverse` function? Read data with it below into a `monarchs` object # The monarchs2 [read_csv("data/__")] and monarchs4 [read_csv2("data/__")] are 'tidyverse function'

2. What is the result of running `class()` on the `monarchs` object created with a tidyverse function. # [1] "spec_tbl_df" "tbl_df" "tbl" "data.frame"

3. How many columns does the object have when created with these different functions? # - `monarchs1` (`read.csv`) and `monarchs2` (`read_csv`) both result in **7 columns** (or the correct number of columns in your dataset) because they use commas `,` as the default delimiter.

- `monarchs3` (`read.csv2`) and `monarchs4` (`read_csv2`) both result in only **1 column** because they expect semicolons `;` as the delimiter, so they fail to split the comma-separated data.

4. Show the dataset so that we can see how R interprets each column

```
# COMPLETE THE BLANKS BELOW WITH YOUR CODE, then turn the 'eval' flag in this chunk t
o TRUE.
library(tidyverse)

monarchs <- read_csv("monarchs_clean.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for deta
ils,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 57 Columns: 7
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr  (2): monarch_name, dynasty
## dbl  (1): monarch_id
## date (4): birth_date, start_of_reign_date, end_of_reign_date, death_date
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
class(monarchs)
```

```
## [1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

```
str(monarchs)
```

```
## spc_tbl_ [57 × 7] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ monarch_id         : num [1:57] 1 2 3 4 5 6 7 8 9 10 ...
##  $ monarch_name       : chr [1:57] "Song_Taizu" "Song_Taizong" "Song_Zhenzong" "So
ng_Renzong" ...
##  $ dynasty            : chr [1:57] "Song" "Song" "Song" "Song" ...
##  $ birth_date         : Date[1:57], format: "0927-03-21" "0939-11-20" ...
##  $ start_of_reign_date: Date[1:57], format: "0960-02-04" "0976-11-15" ...
##  $ end_of_reign_date  : Date[1:57], format: "0976-11-14" "0997-05-08" ...
##  $ death_date         : Date[1:57], format: "0976-11-14" "0997-05-08" ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   monarch_id = col_double(),
##   ..   monarch_name = col_character(),
##   ..   dynasty = col_character(),
##   ..   birth_date = col_date(format = ""),
##   ..   start_of_reign_date = col_date(format = ""),
##   ..   end_of_reign_date = col_date(format = ""),
##   ..   death_date = col_date(format = "")
##   .. )
##  - attr(*, "problems")=<pointer: 0x12168a510>
```

```
glimpse(monarchs)
```

```
## Rows: 57
## Columns: 7
## $ monarch_id          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,…
## $ monarch_name        <chr> "Song_Taizu", "Song_Taizong", "Song_Zhenzong", "So…
## $ dynasty             <chr> "Song", "Song", "Song", "Song", "Song", "Song", "S…
## $ birth_date          <date> 0927-03-21, 0939-11-20, 0968-12-23, 1010-05-30, 1…
## $ start_of_reign_date <date> 0960-02-04, 0976-11-15, 0997-05-08, 1022-03-24, 1…
## $ end_of_reign_date   <date> 0976-11-14, 0997-05-08, 1022-03-23, 1063-04-30, 1…
## $ death_date          <date> 0976-11-14, 0997-05-08, 1022-03-23, 1063-04-30, 1…
```

Calculate the duration of reign for all the monarchs in your table

You can calculate the duration of reign in years with `mutate` function by subtracting the equivalents of your `startReign` from `endReign` columns and writing the result to a new column called `duration`. But first you need to check a few things:

- Is your data messy? Fix it before re-importing to R
- Do your start and end of reign columns contain NAs? Choose the right strategy to deal with them: `na.omit()`, `na.rm=TRUE`, `!is.na()`

Create a new column called `duration` in the monarchs dataset, utilizing the `mutate()` function from tidyverse. Check with your group to brainstorm the options.

```
# YOUR CODE
monarchs <- monarchs %>% filter(!is.na(start_of_reign_date) & !is.na(end_of_reign_dat
e)) %>% mutate(duration = end_of_reign_date - start_of_reign_date)

head(monarchs %>% select(monarch_name, start_of_reign_date, end_of_reign_date, durati
on))
```

```
## # A tibble: 6 × 4
##   monarch_name  start_of_reign_date end_of_reign_date duration
##   <chr>         <date>              <date>            <drtn>
## 1 Song_Taizu    0960-02-04          0976-11-14         6128 days
## 2 Song_Taizong  0976-11-15          0997-05-08         7479 days
## 3 Song_Zhenzong 0997-05-08          1022-03-23         9084 days
## 4 Song_Renzong  1022-03-24          1063-04-30        15012 days
## 5 Song_Yingzong 1063-05-01          1067-01-25         1365 days
## 6 Song_Shenzong 1067-01-26          1085-04-01         6640 days
```

Calculate the average duration of reign for all rulers

Do you remember how to calculate an average on a vector object? If not, review the last two lessons and remember that a column is basically a vector. So you need to subset your `monarchs` dataset to the `duration` column. If you subset it as a vector you can calculate average on it with `mean()` base-R function. If you subset it as a tibble, you can calculate average on it with `summarize()` tidyverse function. Try both ways!

- You first need to know how to select the relevant `duration` column. What are your options?

- Is your selected `duration` column a tibble or a vector? The `mean()` function can only be run on a vector. The `summarize()` function works on a tibble.
- Are you getting an error that there are characters in your column? Coerce your data to numbers with `as.numeric()`.
- Remember to handle NAs: `mean(X, na.rm=TRUE)`

```
# YOUR CODE
monarchs_summary <- monarchs %>% summarise(avg_duration = mean(as.numeric(duration),n
a.rm = TRUE))
```

How many and which monarchs enjoyed a longer-than-average duration of reign?

You have calculated the average duration above. Use it now to `filter()` the `duration` column in `monarchs` dataset. Display the result and also count the resulting rows with `count()`

```
# YOUR CODE
long_reign_monarchs <- monarchs %>% filter(as.numeric(duration) > mean(as.numeric(dur
ation), na.rm = TRUE))

long_reign_monarchs %>% select(monarch_name, start_of_reign_date, end_of_reign_date,
duration)
```

```
## # A tibble: 22 × 4
##    monarch_name  start_of_reign_date end_of_reign_date duration
##    <chr>         <date>              <date>            <drtn>
##  1 Song_Taizong  0976-11-15          0997-05-08         7479 days
##  2 Song_Zhenzong 0997-05-08          1022-03-23         9084 days
##  3 Song_Renzong  1022-03-24          1063-04-30        15012 days
##  4 Song_Shenzong 1067-01-26          1085-04-01         6640 days
##  5 Song_Huizong  1100-02-23          1126-01-18         9460 days
##  6 Song_Gaozong  1127-06-12          1162-07-24        12826 days
##  7 Song_Xiaozong 1162-07-24          1189-02-18         9706 days
##  8 Song_Ningzong 1194-07-24          1224-09-17        11013 days
##  9 Song_Lizong   1224-09-17          1264-11-16        14670 days
## 10 Yuan_Kublai   1260-05-05          1294-02-18        12342 days
## # ℹ 12 more rows
```

```
long_reign_monarchs %>% count(name = "total_above_average_monarchs")
```

```
## # A tibble: 1 × 1
##   total_above_average_monarchs
##                          <int>
## 1                           22
```

How many days did the three longest-ruling monarchs rule?

- Sort monarchs by reign `duration` in the descending order. Select the three longest-ruling monarchs with the `slice()` function

- Use `mutate()` to create `Days` column where you calculate the total number of days they ruled
- BONUS: consider the transition year (with 366 days) in your calculation!

```
# YOUR CODE
top3_monarchs <- monarchs %>%
  arrange(desc(as.numeric(duration))) %>%
  slice(1:3) %>%
  mutate(Days = round(as.numeric(duration)))

top3_monarchs %>%
  select(monarch_name,duration, Days)
```

```
## # A tibble: 3 × 3
##   monarch_name  duration    Days
##   <chr>         <drtn>     <dbl>
## 1 Qing_Kangxi   22595 days 22595
## 2 Qing_Qianlong 22029 days 22029
## 3 Ming_Wanli    17562 days 17562
```

Challenge: Plot the monarchs' duration of reign through time

What is the long-term trend in the duration of reign among Danish monarchs? How does it relate to the historical violence trends ?

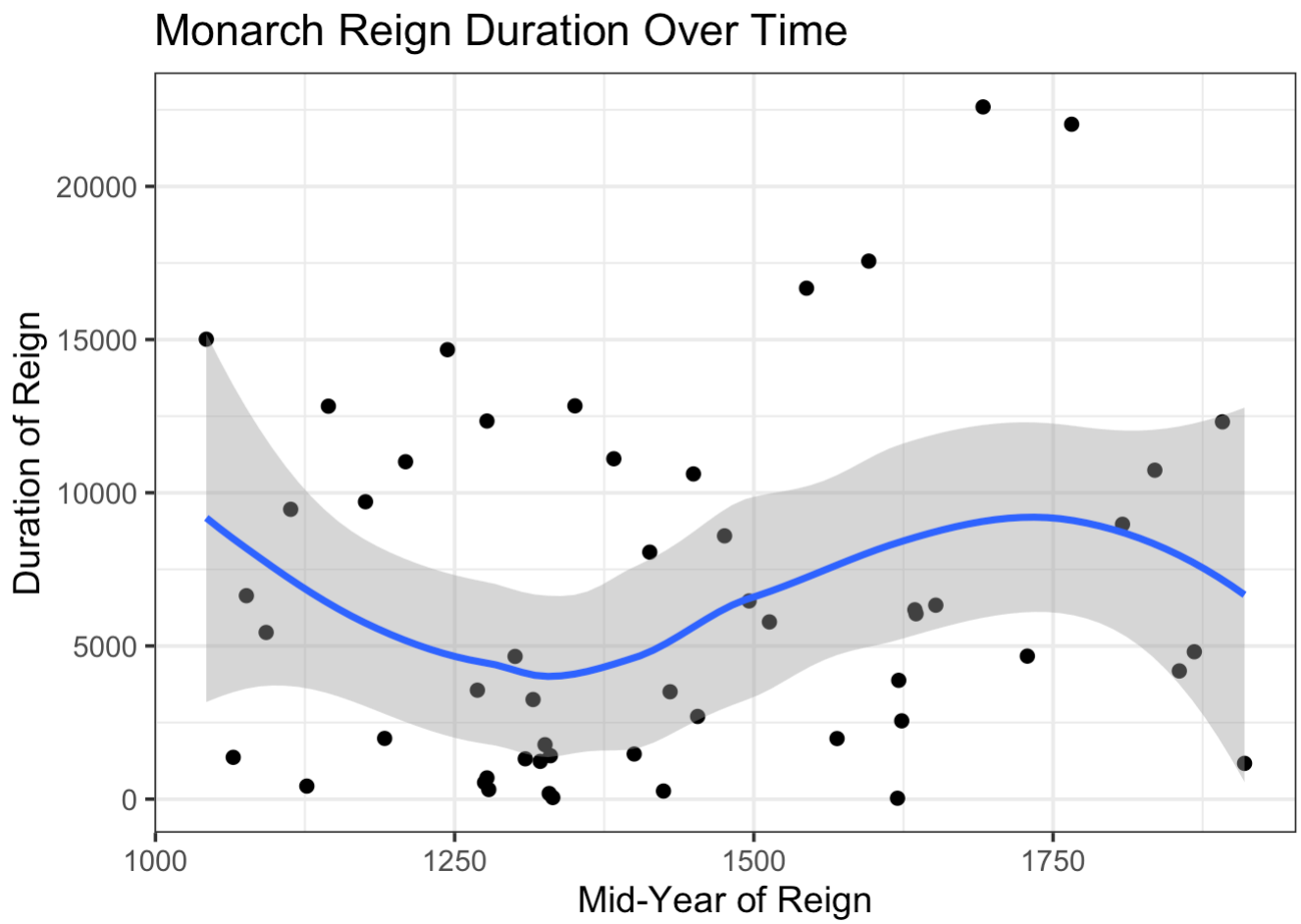
- Try to plot the duration of reign column in `ggplot` with `geom_point()` and `geom_smooth()`
- In order to peg the duration (which is between 1-99) somewhere to the x axis with individual centuries, I recommend creating a new column `midyear` by adding to `startYear` the product of `endYear` minus the `startYear` divided by two (`startYear + (endYear-startYear)/2`).
- Now you can plot the kings dataset, plotting `midyear` along the x axis and `duration` along y axis
- BONUS: add a title, nice axis labels to the plot and make the theme B&W and font bigger to make it nice and legible!

```
# YOUR CODE
monarchs_plot <- monarchs %>%
  mutate(startYear = as.numeric(str_sub(start_of_reign_date, 1, 4)), endYear = as.num
eric(str_sub(end_of_reign_date, 1, 4)), midYear = startYear + (endYear - startYear)/
2, duration = as.numeric(duration)) %>% filter(!is.na(midYear) & !is.na(duration))
```

```
## Warning: There were 2 warnings in `mutate()`.
## The first warning was:
## ℹ In argument: `startYear = as.numeric(str_sub(start_of_reign_date, 1, 4))`.
## Caused by warning:
## ! NAs introduced by coercion
## ℹ Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
ggplot(monarchs_plot, aes(x = midYear, y = duration)) + geom_point() + geom_smooth()
+ theme_bw(base_size = 14) + labs (title = "Monarch Reign Duration Over Time", x = "M
id-Year of Reign", y = "Duration of Reign")
```

```
## `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



And to submit this rmarkdown, knit it into html. But first, clean up the code chunks, adjust the date, rename the author and change the `eval=FALSE` flag to `eval=TRUE` so your script actually generates an output. Well done!